

1. What are Distributed Systems? Explain their key characteristics with examples.

Answer:

A **Distributed System** is a collection of independent computers that appear to the users as a single coherent system. These computers communicate and coordinate their actions by passing messages over a network. The key characteristics of distributed systems are:

Key Characteristics:

1. **Resource Sharing** – Resources such as files, printers, or computational power can be shared among users.
2. **Concurrency** – Multiple processes may run concurrently, interacting and sharing resources.
3. **Scalability** – A distributed system can handle increased loads by adding more nodes.
4. **Fault Tolerance** – The system should continue operating even if some components fail.
5. **Transparency** – Users should not be aware of the underlying system complexity.

Examples of Distributed Systems:

- **The World Wide Web (WWW):** The web consists of servers distributed across the world, serving content to users via the HTTP protocol.
- **Cloud Computing (AWS, Google Cloud):** Provides storage, computing, and networking resources on a large scale.
- **Blockchain Networks:** Bitcoin and Ethereum use a distributed ledger that operates without a central authority.

2. What are the challenges in designing and implementing distributed systems?

Answer:

Developing a distributed system presents several challenges due to the nature of distributed computing. These include:

1. **Heterogeneity** – Different hardware, operating systems, and network protocols must interoperate.
2. **Fault Tolerance** – The system should continue functioning despite node failures or network issues.
3. **Concurrency** – Multiple users or processes should be able to work simultaneously without conflicts.
4. **Scalability** – The system should handle an increasing number of users, data, and resources.
5. **Security** – Ensuring authentication, authorization, and encryption is essential to prevent attacks.
6. **Synchronization** – Keeping data consistent across multiple nodes is challenging, requiring techniques like distributed consensus.

7. **Transparency** – Users should not be aware of system failures, data migration, or replication.

To overcome these challenges, distributed systems use replication, fault detection mechanisms, and security protocols.

3. Explain the different architectural models used in distributed systems.

Answer:

Architectural models define how distributed system components interact. The common models are:

1. **Client-Server Model**

- Clients request services from servers, which process and respond.
- Examples: Web applications (browsers as clients, web servers as servers).

2. **Peer-to-Peer (P2P) Model**

- No central authority; all nodes act as both clients and servers.
- Examples: BitTorrent, Blockchain networks.

3. **Three-Tier Model**

- Splits processing into three layers: Presentation (UI), Application (Logic), and Database.
- Examples: Web applications with front-end, middleware, and backend databases.

4. **Event-Driven Model**

- Components react to events instead of direct communication.
- Examples: IoT systems, message queues like Kafka.

5. **Microservices Architecture**

- Applications are composed of small, independent services communicating over APIs.
- Examples: Netflix, Amazon Web Services.

4. What is interprocess communication (IPC) in distributed systems? Explain its types with examples.

Answer:

Interprocess Communication (IPC) enables processes in a distributed system to communicate and synchronize their actions.

Types of IPC:

1. **Message Passing:**

- Processes send and receive messages over a network.

- Example: HTTP requests, RPC (Remote Procedure Call).
- 2. **Shared Memory:**
 - Processes communicate by reading/writing to a shared memory space.
 - Example: POSIX Shared Memory in UNIX.
- 3. **Pipes and Named Pipes:**
 - Unidirectional or bidirectional data streams between processes.
 - Example: Linux pipe() system call.
- 4. **Sockets:**
 - Communication over network protocols such as TCP/IP.
 - Example: A web browser retrieving a page from a web server.
- 5. **Remote Procedure Calls (RPC):**
 - Allows invoking procedures on remote systems as if they were local.
 - Example: gRPC (Google RPC).

5. What is marshalling and external data representation in distributed systems?

Answer:

Marshalling is the process of converting an object or data structure into a format suitable for storage or transmission.

External Data Representation (EDR)

Since different systems use different data formats (e.g., little-endian vs. big-endian), external data representation ensures compatibility.

Common methods include:

1. **XDR (External Data Representation):** Used in Sun RPC for platform-independent data exchange.
2. **JSON & XML:** Used for web APIs due to human-readability.
3. **Protocol Buffers (protobufs):** Google's efficient binary format.

Example:

- A Java client serializes an object into JSON and sends it to a Python server, which deserializes it.

6. How does client-server communication work in distributed systems?

Answer:

Client-server communication follows a request-response model where:

1. The client sends a request to the server.
2. The server processes the request and returns a response.

Protocols Used:

1. **HTTP/HTTPS:** Web applications.
2. **Remote Procedure Call (RPC):** gRPC, XML-RPC.
3. **Sockets:** Low-level communication (TCP/UDP).

Example:

- A browser (client) sends an HTTP request to a web server, which responds with an HTML page.

7. Explain group communication in distributed systems.

Answer:

Group communication involves multiple processes communicating as a group.

Types:

1. **Multicast Communication:** One sender, multiple receivers (e.g., live streaming).
2. **Broadcast:** Message sent to all nodes in a network.
3. **Reliable Group Communication:** Guarantees message delivery to all group members.

Example:

- **Google Meet:** Uses multicast to deliver video streams to participants.

8. How does UNIX handle interprocess communication (IPC)?

Answer:

UNIX provides several IPC mechanisms:

1. **Pipes:** Used for communication between parent and child processes.
2. **Message Queues:** Allows structured message passing.
3. **Shared Memory:** Enables multiple processes to access a common memory region.
4. **Sockets:** Network-based IPC between different machines.

Example:

c

CopyEdit

```
// Using Pipes in UNIX
```

```
int fd[2];
```

```
pipe(fd);
```

```
write(fd[1], "Hello", 5);
```

```
read(fd[0], buffer, 5);
```

This code sets up a pipe where one process writes data, and another reads it.

9. What is Remote Procedure Call (RPC) and how does it work?

Answer:

RPC allows a program to execute a function on a remote server as if it were local.

Working of RPC:

1. Client calls a function.
2. Stub (client-side) serializes the request.
3. Message is sent over the network.
4. Server stub receives, deserializes, and calls the function.
5. Response follows the reverse path.

Example:

- gRPC is used in microservices for efficient RPC communication.

10. What are the key differences between TCP and UDP for interprocess communication?

Answer:

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable (acknowledgments)	Unreliable
Speed	Slower	Faster
Use Cases	Web, File Transfer	Streaming, Gaming

Example:

- A video call uses **UDP** to minimize delay, while a file transfer uses **TCP** for reliability.

